

# AI CR 评估工程

---

引言

核心

指标体系

关键指标数据统计

关键指标

指标定义

混淆矩阵

计算公式

统计难点

AI CR 采纳数据自动收集

本地模式

现有业务分析

MCP 服务分批次 CR 流程

AI CR 覆盖率分析机制：

整体思路

整体流程

整体实现

基于 AI CR 建议代码字符串匹配

压缩前

压缩后

基于 AI CR 建议代码 AST 语义片段的精确对比 - 对比方案

两种方案的对比

GitLab Web Hooks 统计 AI CR Issue 列表

数据模型

MR 模式

数据流程图

数据模型

人工 CR 采纳数据收集

[采纳判断方案](#)

[数据流程图](#)

[GitLab Discussions API 返回数据分析](#)

[数据模型设计](#)

[Bug 数据收集](#)

[数据收集](#)

[Bug 数据结构分析](#)

[存储表结构设计](#)

[Bug 同步流程](#)

[数据清理](#)

[清理流程](#)

[清理结果分类](#)

[判定规则配置](#)

[实际 Case 说明](#)

[人工复核机制](#)

[数据归属](#)

[判定规则](#)

[Case 说明](#)

[边界场景处理](#)

[遗留问题](#)

[展示页面](#)

[参考](#)

## 引言

当你对一个提示词进行修改后，你如何科学地知道，这次修改是让 AI 应用变好了，还是变坏了？

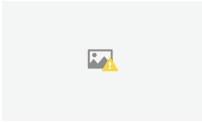
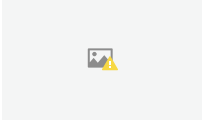
“感觉”是不可靠的。评估与优化，就是一门“数据驱动的科学”的系统性方法论。

## 核心

**核心理念：**用数据而非感觉来驱动优化。评估是将一个模糊的、主观的“好”，分解成一系列清晰的、可量化的指标 (Metrics) 的过程。

关键步骤：构建评估数据集（benchmark + 测试集）、定义评估指标（度量体系）、运行评估与优化（成熟方案：LLM\_as\_a\_judge）。

## 指标体系

| 分类  | 指标  | 计算公式                                                                                                                                                                                | 收集方式                                                                                                                                                                                           | 数据来源                         | 目标                |
|-----|-----|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------|-------------------|
| 效果类 | 采纳率 | AI 被采纳建议数 / AI 建议总数<br><br><code>accepted_ai_comments</code> / <code>all_ai_comments</code>                                                                                         | 每条 AI 建议记录的用户采纳状态：采纳、不采纳、未反馈                                                                                                                                                                   | 外化平台                         | 衡量 AI CR 是否准确     |
|     | 召回率 | AI 被采纳建议数 / （AI 被采纳建议数 + 人工被采纳建议数）<br><br><code>accepted_ai_comments</code> / ( <code>accepted_ai_comments</code> + <code>accepted_humman_comments</code> + <code>all_bugs</code> ) | 每条 AI 建议记录的用户采纳状态：采纳、不采纳、未反馈<br><br>每条人工建议记录的采纳状态：采纳、不采纳、未反馈                                                                                                                                   | 外化平台<br>Gitlab 平台<br>Jira 平台 | 衡量 AI 审查是否能代替人工审查 |
|     | 好评率 | 用户反馈好评数 / 用户反馈总数<br><br><code>like_reviews</code> / <code>all_reviews</code>                                                                                                        | 每次审查记录的用户反馈：<br> 、<br> | 外化平台                         | 衡量用户对产品的认可度       |

|     |           |                                                                                                              |                                                                 |           |                                      |
|-----|-----------|--------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------|-----------|--------------------------------------|
| 使用类 | 需求覆盖率     | 使用了 AI CR 的 MR 数 / MR 总数<br><br><code>mrs_with_ai_cr</code> / <code>all_mrs</code>                           | MR 合入分支是否做过 AI CR                                               | Gitlab 平台 | 衡量 AI CR 在研发流程中的覆盖程度                 |
|     | MR 代码未覆盖率 | 未覆盖的提交数 / 所有提交总数<br><br><code>commits_with_ai_cr</code> / <code>all_ai_commits</code>                        | MR 合入 commit 是否做过 AI CR                                         | Gitlab 平台 | 衡量哪些代码变化在无 AI CR 情况下直接进入了主干，属于风险暴露指标 |
|     | 用户活跃度     | 用一次人数/深度使用人数/总人数<br><br><code>once_users</code> / <code>more_than_once_users</code> / <code>all_users</code> | 用过一次：单位时间内，调用次数等于1<br><br>深度使用：单位时间内，调用次数大于1<br><br>总人数：已注册研发人数 | mcp       | 衡量研发用户的使用程度                          |
|     | 高危建议占比    | 高危建议总数 / 建议总数<br><br><code>score_lte_4_ai_comments</code> / <code>all_ai_comments</code>                     | 高危建议：评分大于等于 4 分（评分等级：2, 3, 4, 5）                                | mcp       | 衡量团队代码质量趋势                           |
| 有效性 | 规则活跃度     | 某条规则触发总数 / AI 建议总数<br><br><code>triggered_rules</code> / <code>all_ai_comments</code>                        | 每次评论是否触发已有的审查规则                                                 | mcp       | 衡量审查规则的质量                            |

|                |                                                                                                                                                           |                 |         |                       |
|----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------|---------|-----------------------|
| 任务完成率          | 完成的审查任务数 / 审查任务总数<br><br><code>complete_reviews</code> / <code>all_reviews</code>                                                                         | 审查流程完结的任务数      | mc<br>p | 衡量审查流程完整性             |
| 质检失败率          | 质检失败次数 / 质检总次数<br><br><code>failed_quality_inspection_times</code> / <code>quality_inspection_times</code>                                                | AI 建议不通过质检的数量   | mc<br>p | 衡量审查建议的质量             |
| 审查任务平均耗时       | 完成的审查任务总耗时 / 完成的审查任务数<br><br><code>complete_reviews_ms_times</code> / <code>complete_reviews</code>                                                       | 审查流程完结的总耗时 (ms) | mc<br>p | 衡量审查流程的效率             |
| 平均100行代码提出建议总数 | 完成的审查任务提出的建议总数 / 完成的审查任务的代码总行数 * 100<br><br><code>ai_comments_in_complete_reviews</code> / <code>code_lines_in_complete_reviews</code> * <code>100</code> | 审查流程完结的产生建议     | mc<br>p | 衡量 AI 审查的有效性与代码质量问题密度 |

## 关键指标数据统计

# 关键指标

## 指标定义

|                |                  |                             |
|----------------|------------------|-----------------------------|
| Precision（采纳率） | AI 发现的问题中，被采纳的比例 | AI 发现的问题大多是准确的，误报少          |
| Recall（召回率）    | 真实问题中，AI 发现的比例   | AI 能发现大部分真实问题，漏报少           |
| F1-Score       | 精确率和召回率的调和平均     | 既能准确识别问题（少误报），又能全面发现问题（少漏报） |

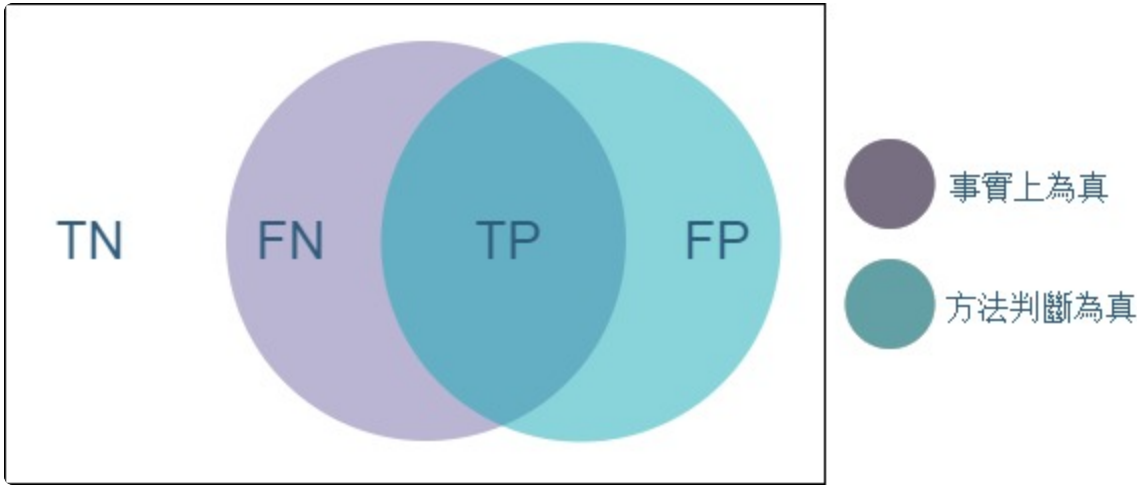
注意：F-Score 是 精确率（Precision/采纳率） 和 召回率（Recall） 的调和平均数，能够综合评估 AI CR 系统的整体表现。这里，我们采用了 F1-Score。

F1-Score 越高 = AI CR 系统越"又准又全"

- "准": 报告的问题大多是真问题（高 Precision）
- "全": 真问题大多能被发现（高 Recall）

## 混淆矩阵

在计算采纳率、召回率、F1-Score 前，需要了解



在这里，TP、FN、FP、TN 分别表示

|                          |                                                                                                            |                            |
|--------------------------|------------------------------------------------------------------------------------------------------------|----------------------------|
| TP (True Positive, 真阳性)  | <p>含义: AI CR 指出了一个问题, 用户采纳了 (或确认这确实是个问题)</p> <p>代表: AI 正确识别出的真实问题</p> <p>示例: AI 说"这里有空指针风险", 开发者确认并修复了</p> | 用户点击"采纳"/"应用修复", 或被自动识别了采纳 |
| FP (False Positive, 假阳性) | <p>含义: AI CR 指出了一个问题, 但用户拒绝/忽略 (认为这不是问题)</p> <p>代表: AI 的误报</p> <p>示例: AI 说"变量命名不规范", 但开发者认为命名是合理的</p>      | 用户点击"忽略"/"拒绝", 或被自动识别了不采纳  |
| FN (False Negative, 假阴性) | <p>含义: AI CR 没有指出某个代码问题, 但实际上代码确实有问题</p> <p>代表: AI 的漏报</p> <p>示例: 代码有 SQL 注入风险, 但 AI 没有检测出来</p>            | 人工代码审查/Test bug 发现 AI 漏检   |
| TN (True Negative, 真阴性)  | <p>含义: AI CR 没有指出问题, 代码确实没有问题</p> <p>代表: AI 正确判断为无问题的代码</p> <p>示例: 代码质量良好, AI 也没有报告任何问题</p>                | 无问题代码且 AI 未报告 – 不需要统计      |

## 计算公式

|                 |                                                             |
|-----------------|-------------------------------------------------------------|
| Precision (采纳率) | $TP / (TP + FP)$                                            |
| Recall (召回率)    | $TP / (TP + FN)$                                            |
| F1-Score        | $2 \times (Precision \times Recall) / (Precision + Recall)$ |

## 统计难点

根据上面的分析，我们需要特别关注的的数据有 TP、FP、FN。

其中 TP、FP 是指 AI CR 发现的问题是否采纳的数据，目前分成两部分收集方式：

1. 通过用户在管理平台手动上报 — **DONE**
2. 通过 AI 建议修复的代码是否仍保留在最终代码中，判断问题最终是否被采纳 — **TODO**

由于 AI CR 支持的模式多样，需要拆成以下 2 种方式来收集：

1. 本地模式：用户对未合入主分支的代码进行审查，AI CR 发现的问题会在本地 IDE 即时修复掉 — **TODO**
2. MR 模式：用户对 MR 的代码进行审查，AI CR 发现的问题会被评论到 MR 上，用户在本地 IDE 修复完问题后，状态会同步到 MR 上 — **TODO**

而 FN 是非 AI 发现的其他真实问题，一般由以下两部分组成：

1. 人工代码审查：由 Reviewer 提出问题，最终被 Dev 采纳的问题会被计入 — **TODO**
2. Test Bug：选择由 QA 提出的 Test Bug，经过清洗后的部分 Bug 会被计入 — **TODO**

## AI CR 采纳数据自动收集

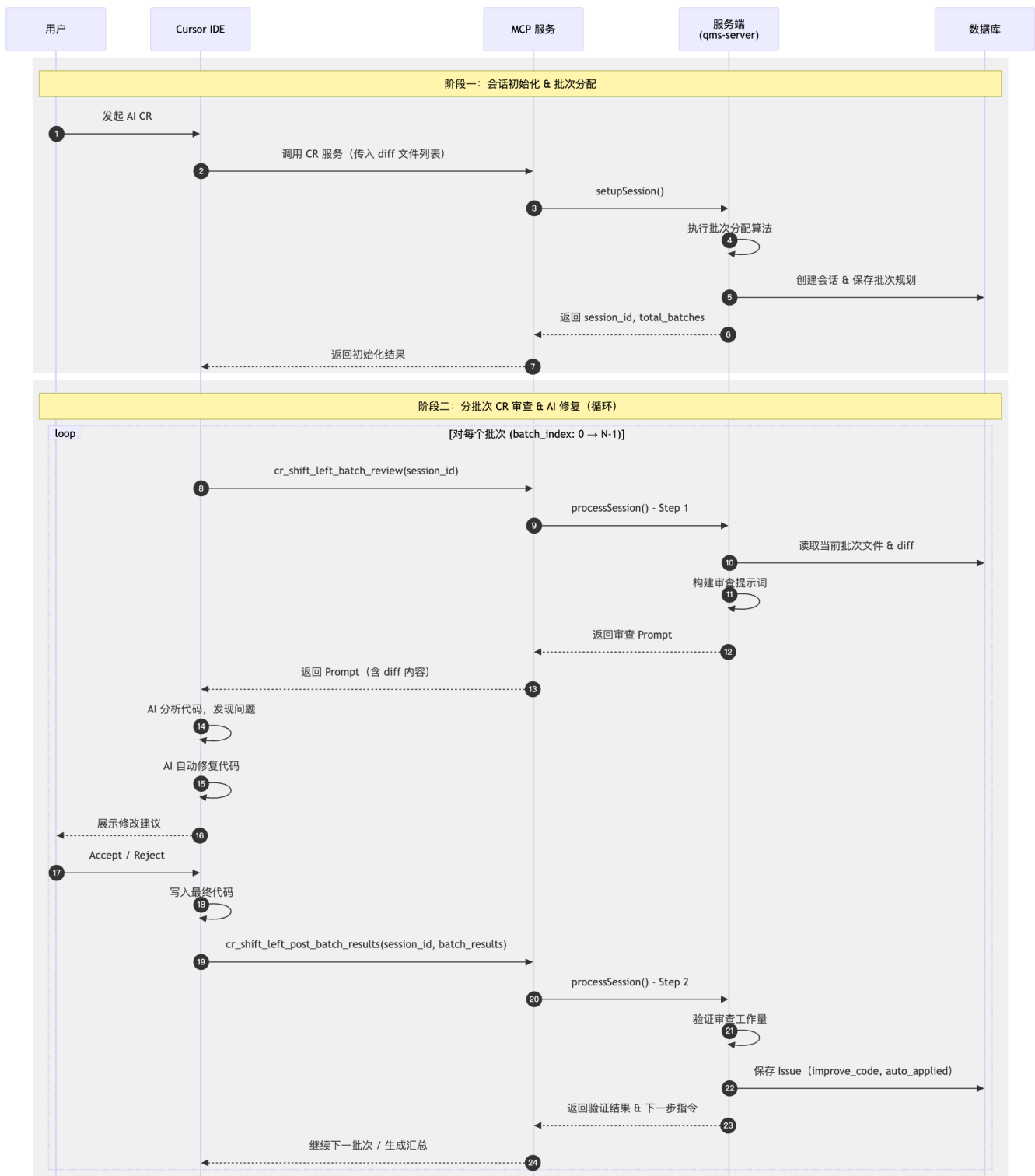
### 本地模式

#### 现有业务分析

#### MCP 服务分批次 CR 流程

整体流程图：





关键数据存储：

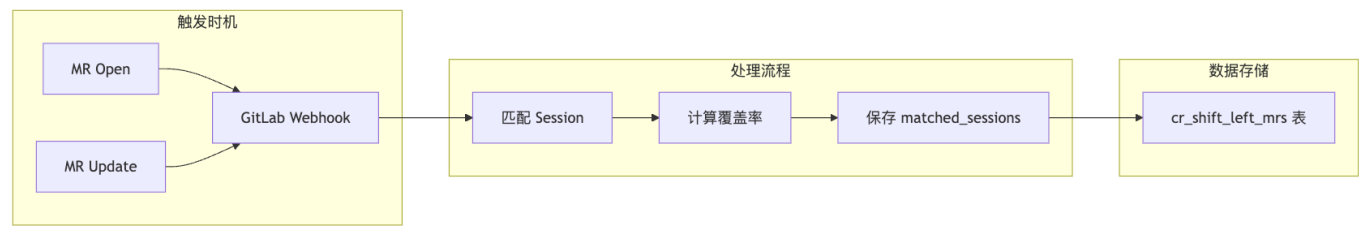
MCP 服务和服层在 CR 过程中会将一下关键数据存储到 cr\_shift\_left\_issues 表中：

|               |                |
|---------------|----------------|
| session_id    | 问题关联的 CR 会话 ID |
| file_path     | 问题所在的文件路径      |
| existing_code | 问题原始代码         |

|              |                       |
|--------------|-----------------------|
| improve_code | AI 建议的修复代码（采纳检测的核心依据） |
| is_applied   | 问题采纳状态                |

AI CR 覆盖率分析机制：

AI CR 覆盖率会在 MR 创建、更新时，自动计算覆盖率，并将匹配结果存储到 cr\_shift\_left\_mrs 表。流程图如下：



关键数据存储：

|                  |                        |
|------------------|------------------------|
| matched_sessions | AI CR 记录中覆盖的 commit 信息 |
|------------------|------------------------|

整体思路

**基本原理：** 利用 MCP 服务 CR 代码时，会将 AI CR 产生的问题和修复建议存在到数据库中，后续在用户合并代码时，检测文件中是否包含 AI 修复建议。

**核心思路：** 采纳 = Merge 后的文件内容包含 AI 建议的 improve\_code 代码片段。

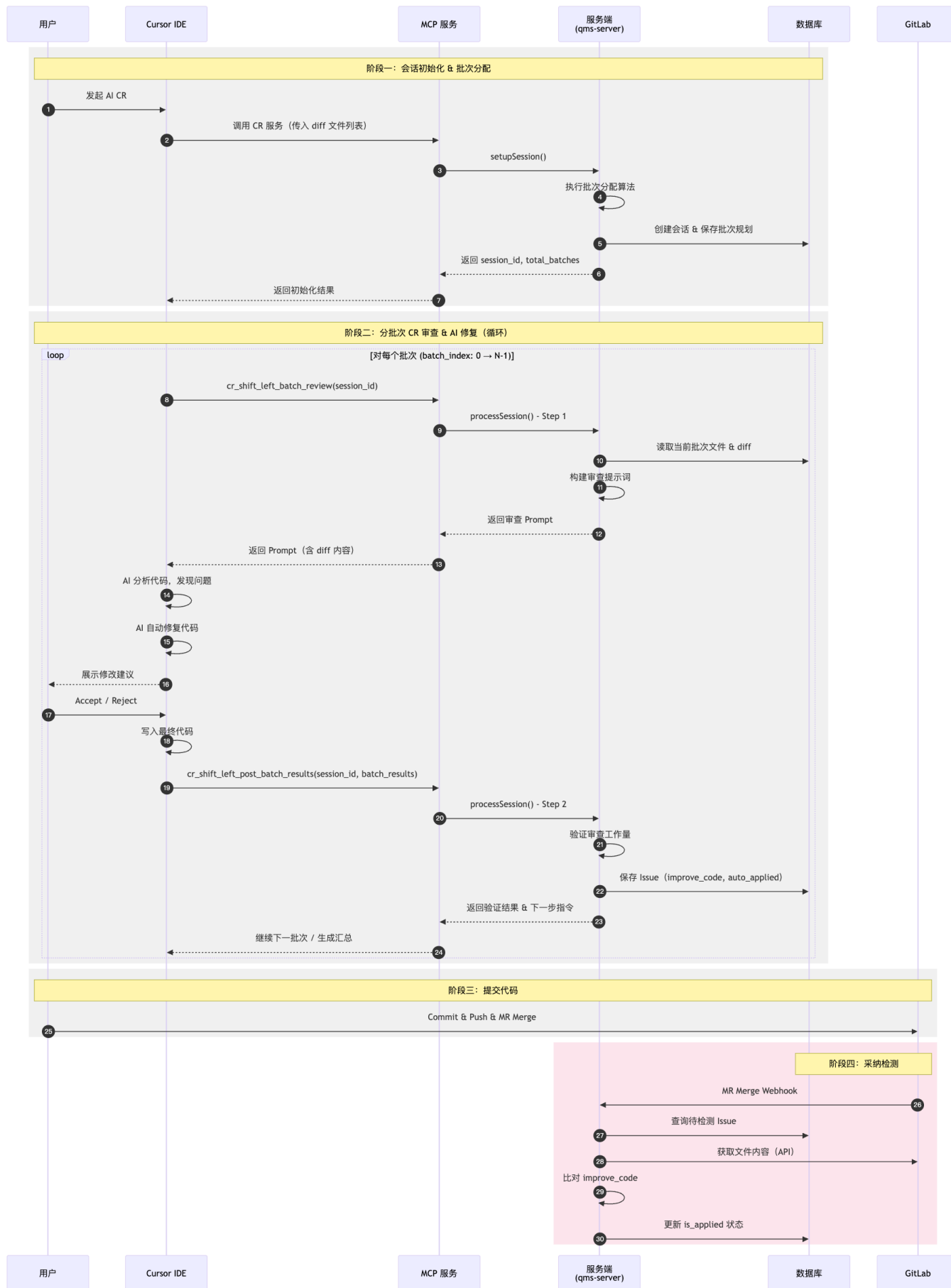
**核心问题：** 在检测时，如何比较判断 AI CR 建议代码存在于当前文件内容中，从而判断 issue 是否被采纳。

整体流程

整体流程概览，粉红色模块为本次方案中涉及的变更点。

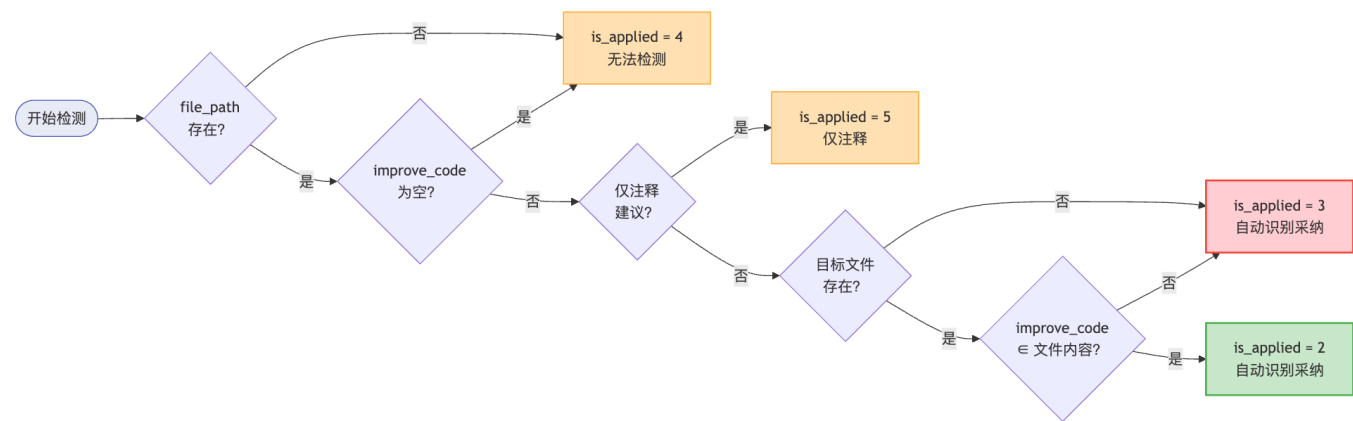
**阶段一：** 用户触发 ibs cr，调用 AI CR Otterlyzee MCP 服务，分批次轮询 CR 问题，并记录产生的 issue。

**阶段二：** 在工程项目上配置 GitLab Web Hooks。用户在 gitLab 上合并代码时，触发 Merge 类型的 Web Hooks 请求，服务端通过 gitLab API 获取对应文件内容，调用采纳检测逻辑，更新数据库中 issue 的状态。



整体实现

检测流程图解析：



|                         |                               |
|-------------------------|-------------------------------|
| file_path 是否存在          | 确认 Issue 记录中是否有文件路径信息         |
| improve_code 是否为空       | 确认 AI 是否提供了具体的修复建议            |
| 是否为仅注释建议                | 确认 AI 的建议是否只是添加/修改注释，没有实际代码变更 |
| 目标文件是否存在                | 确认 MR Merge 后，目标文件是否还存在于代码仓库中 |
| improve_code 是否包含在文件内容中 | 核心检测逻辑，判断 AI 建议的代码是否被用户保留了    |

基于 AI CR 建议代码字符串匹配

**代码压缩策略：**为什么要进行代码压缩？因为 AI CR 生成的建议代码，只是一个代码片段，而真实的代码环境是有缩进、空格、空行等格式化差异的，如果直接进行字符串比较会失败。

- 去除空格符
- 去除划行
- 移除空行

压缩前

```
1 // improve code
2 if(data) {
3     returndata.value;
4 }
5
6 // user current code
7 import{ Data } from './types';
8
9 export function getValue(data) {
10     if(data) {
11         returndata.value;
12     }
13     returnnull;
14 }
15 // improve code
16 if(data){returndata.value;}
17
18 // user current code
19 import{Data}from'./types';exportfunctiongetValue(data){if(data){returndata
    .value;}returnnull;}
```

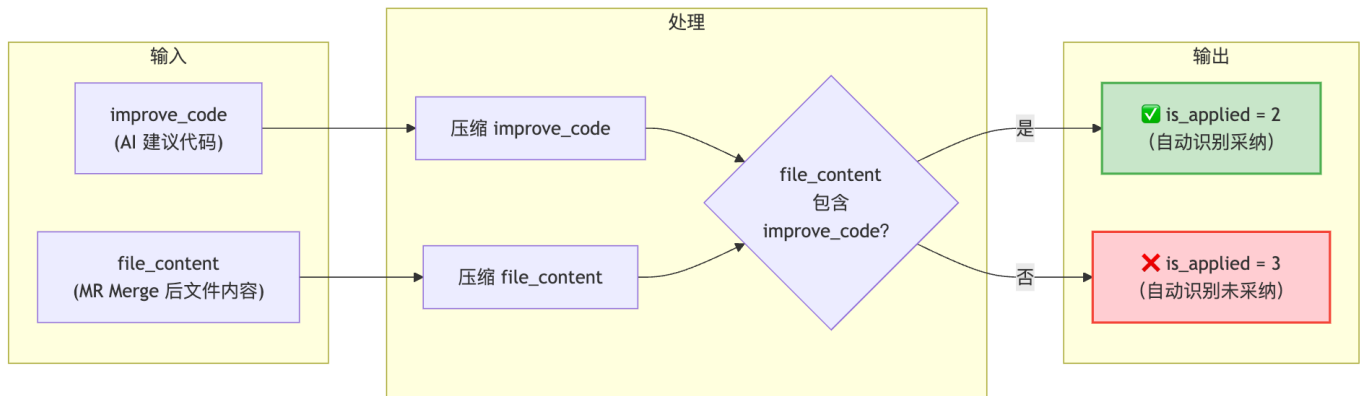
## 压缩后

```
1 // improve code
2 if(data){returndata.value;}
3
4 // user current code
5 import{Data}from'./types';exportfunctiongetValue(data){if(data){returndata.
    value;}returnnull;}
```

代码压缩主要解决的是缩进、空行以及格式化工具导致的换行位置变化等问题。

**方案概述：**利用 AI CR 生成的建议代码，进行字符串代码压缩后，对代码串进行精确比对。

**核心思路：**在 MR Merge 时，通过 GitLab API 获取目标分支的文件内容，检查是否包含 AI 建议的 improve\_code 的代码。



### 基于 AI CR 建议代码 AST 语义片段的精确对比 – 对比方案

**方案概述：**AST（Abstract Syntax Tree，抽象语法树）比较方案通过解析代码的语法结构来判断采纳，而非简单的字符串匹配。这种方式可以忽略格式差异，直接比较代码的语义是否一致。

**核心思路：**只检查 AI CR 建议代码的语义特征是否存在于代码中。

**AST 比较的核心原则：**AST 不同就是不同，不做语义等价规则判断。语义等价判断复杂度较高、难以穷举、容易产生误判。

| AST 结构完全一致                                                                                                                                                                                                                       | AST 结构不同                                                                                                                                                                                                                                                                                                                                       |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <div>▼</div> <pre> 1  // improve_code (AI 建议) 2  constvalue = data?.value    ?? 'default'; 3 4  // current_code (用户实际代码, 格式不同) 5  constvalue = data?.value 6    ?? 'default'; 7 8  // AST 比较结果: 9  // 两者的 AST 结构完全一致    ✅ </pre> | <div>▼</div> <pre> 1  // improve_code (AI 建议) 2  constvalue = data?.value ?? 'default'; 3 4  // current_code (用户使用不同写法) 5  constvalue = data &amp;&amp; data.value ? data    .value : 'default'; 6 7  // AST 比较结果: 8  // AST 结构不同 (OptionalChaining vs LogicalExpression) 9  // → 判断为未采纳 ❌ 10 // 说明: 虽然语义可能等价, 但 AST 不同就是不同, 不做语义等价规则判断 </pre> |

**AST 方案的实现要点：**

- 语言支持与解析器(关键)

AST 方案的核心前提是：目标语言必须有可用的 AST 解析器。这是 AST 方案的关键限制。

|                    |                                            |                                   |
|--------------------|--------------------------------------------|-----------------------------------|
| TS / JS            | @typescript-eslint/parser<br>@babel/parser | ✅ 成熟稳定                            |
| Go / Python / Java | npm install tree-sitter                    | ✅ 语言本身支持转为 AST，但在 node 环境下要引入通用的库 |
| SQL / Shell / Bash | -                                          | ❌ 差异大，基本无法支持                      |

结论：不是所有语言都成熟的 AST 解析器，小语言支持有限，同时服务端要预装多种解析器，成本高，容错率低。

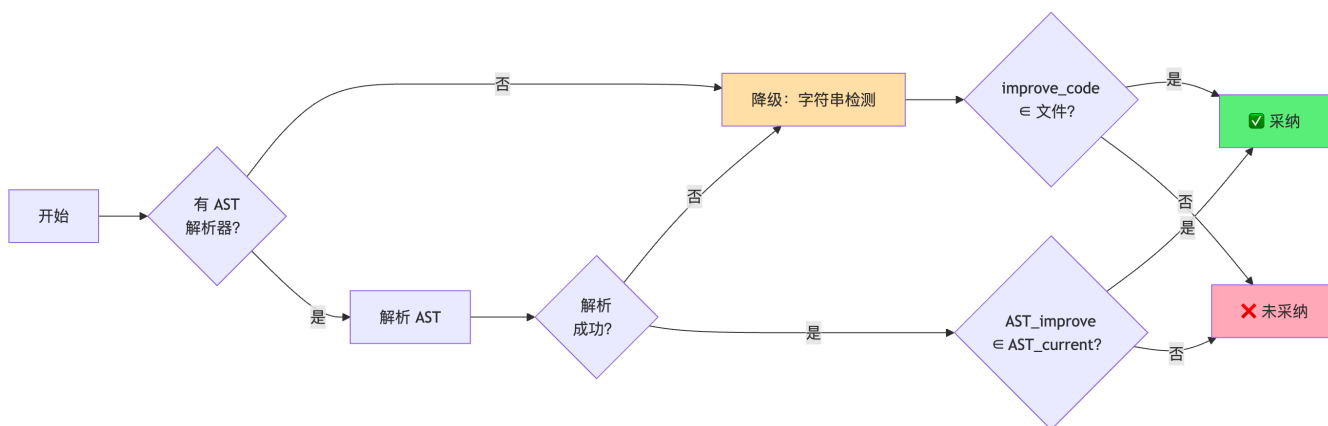
- 降级策略

不支持 AST 时进行降级判断，构建一个文件类型和语言的映射表，判断到不支持 AST 解析时，采用字符串比对的方式。

- 需要对现有 AI CR 生成的建议代码改造

当前 AI CR 生成的建议代码并不是严格的代码片段，可能是字符串，这种情况下无法构建 AST，需要进行降级处理。

AST 方案的判断流程图：



两种方案的对比

|    |                                   |                                              |
|----|-----------------------------------|----------------------------------------------|
| 原理 | 检查 MR Merge 后的文件是否包含 AI CR 建议代码片段 | 解析代码为 AST，检查 MR Merge 后的文件是否存在 AI CR 建议的语义片段 |
|----|-----------------------------------|----------------------------------------------|

|         |                              |                                                          |
|---------|------------------------------|----------------------------------------------------------|
| 准确性与复杂度 | 准确性高，复杂度低                    | 准确性极高，但实现起来很复杂，成本很高                                      |
| 局限性     | 1. 无法处理注释差异                  | 1. 兼容性较差，AST 解析器无法完全覆盖所有语言，对小众语言支持不够友好<br>2. 处理大型文件时性能较差 |
| 优势性     | 1. 成本低<br>2. 无语言依赖，适用于所有文件类型 | 1. AST 无注释差异，注释不进入 AST                                   |

## GitLab Web Hooks 统计 AI CR Issue 列表

**触发时机：**经过考虑，我们选择在用户提交 MR 被合并时，触发采纳率检测。原因如下：

- 准确性：这个时机可以拿到用户文件内容的最终状态。
- 自动化：用户无感知，自动触发。
- 实时性：MR 被合并后立即触发。
- 兼容性：兼容性强，不侵入项目配置，只通过 gitlab web hook 配置实现。

**issue 关联与过滤策略：**当 GitLab Web Hook 触发时，如果确定哪些 Issue 需要进行采纳检测？

核心思路：通过数据库中存储的 MR 记录，在覆盖率功能中，如果当前 MR 进行过 CR，则会存储对应的 CR 记录(cr\_shift\_left\_mrs 表中 matched\_sessions 字段)，通过 Session 记录获取到对应的 Issue 列表。

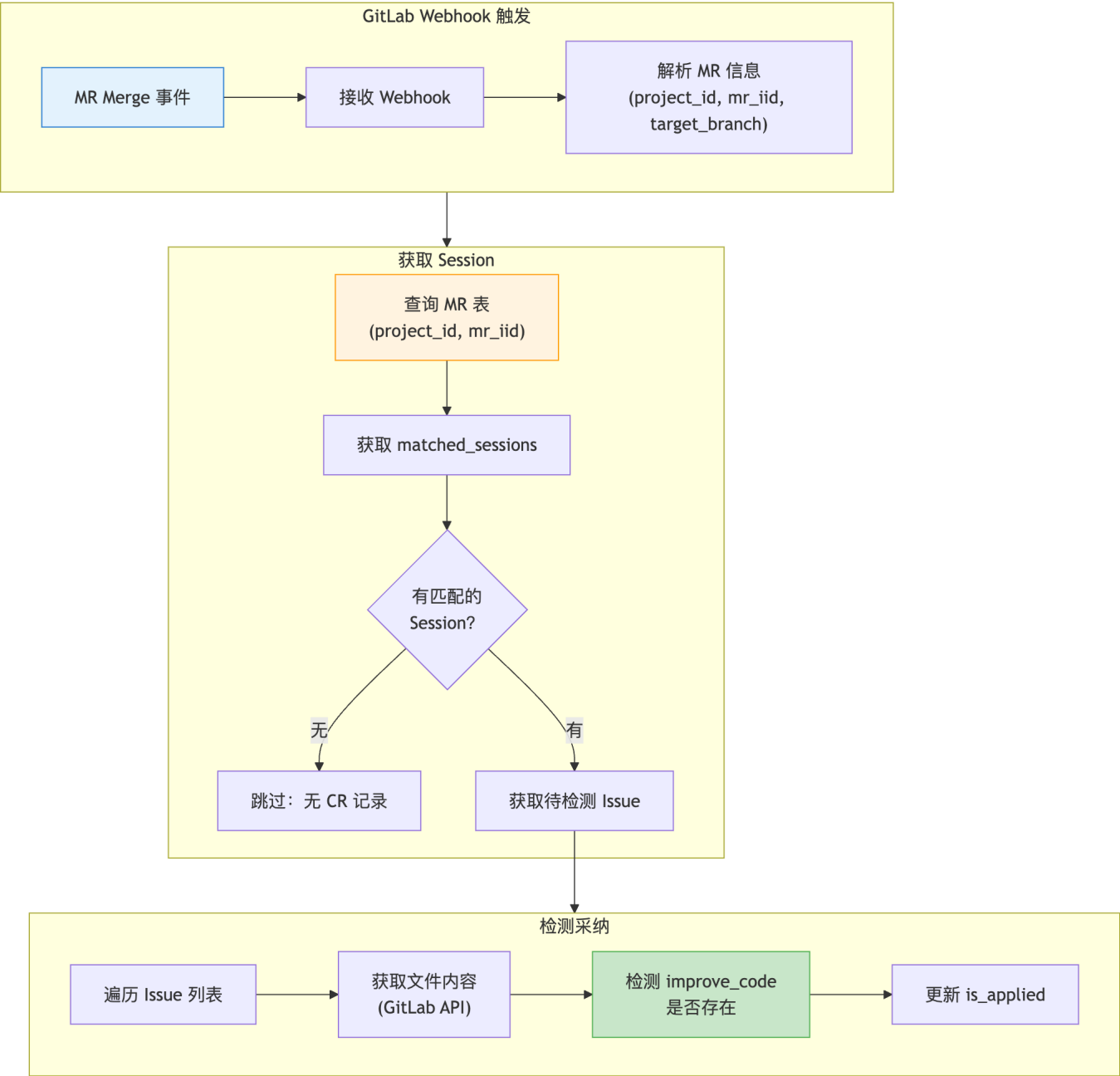
- GitLab Web Hooks 发起时，Request 请求会传递仓库和 MR 的基本信息，例如：源分支、目标分支、项目 ID、MR ID等。
- 因此我们可以通过查询获取到对应的 Session\_ids 后，进行联表查询出对应的待检测 issue 列表。

**关键 API 梳理：**



```
1 通过 GitLab API 获取文件内容
2 GET /projects/:id/repository/files/:file_path?ref=:branch
3
4 // Response
5 {
6   "file_name": "key.rb",
7   "file_path": "app/models/key.rb",
8   "size": 1476,
9   "encoding": "base64",
10  "content": "IyA9PSBTY2h1bWEgSW5mb3...",
11  "content_sha256": "4c294617b60715c1d218e61164a3abd4808a4284cbc30e6728a01
    ad9aada4481",
12  "ref": "main",
13  "blob_id": "79f7bbd25901e8334750839545a9bd021f0e4c83",
14  "commit_id": "d5a3ff139356ce33e37e73add446f16869741b50",
15  "last_commit_id": "570e7b2abdd848b95f2f578043fc23bd6f6fd24d",
16  "execute_filemode": false
17 }
```

完整流程图如下：



数据模型

issue 数据模型 is\_applied 字段改造:

|      |           |           |
|------|-----------|-----------|
| NULL | 未反馈，用户未操作 | 未反馈，用户未操作 |
| 0    | 人工标记未采纳   | 人工标记未采纳   |
| 1    | 人工标记已采纳   | 人工标记已采纳   |
| 2    | -         | 自动识别已采纳   |

|   |   |                                        |
|---|---|----------------------------------------|
| 3 | - | 自动识别未采纳                                |
| 4 | - | 无法自动检测， improve_code 为空，例如：AI 建议删除冗余代码 |
| 5 | - | 仅注释建议， AI 只是新增/修改注释，没有代码逻辑变更           |

采纳率统计时， is\_applied = 1 或 is\_applied = 2 都视为采纳。

不参与计算： NULL（未检测）、4（无法检测）、5（仅注释建议）。

issue 数据模型扩展：

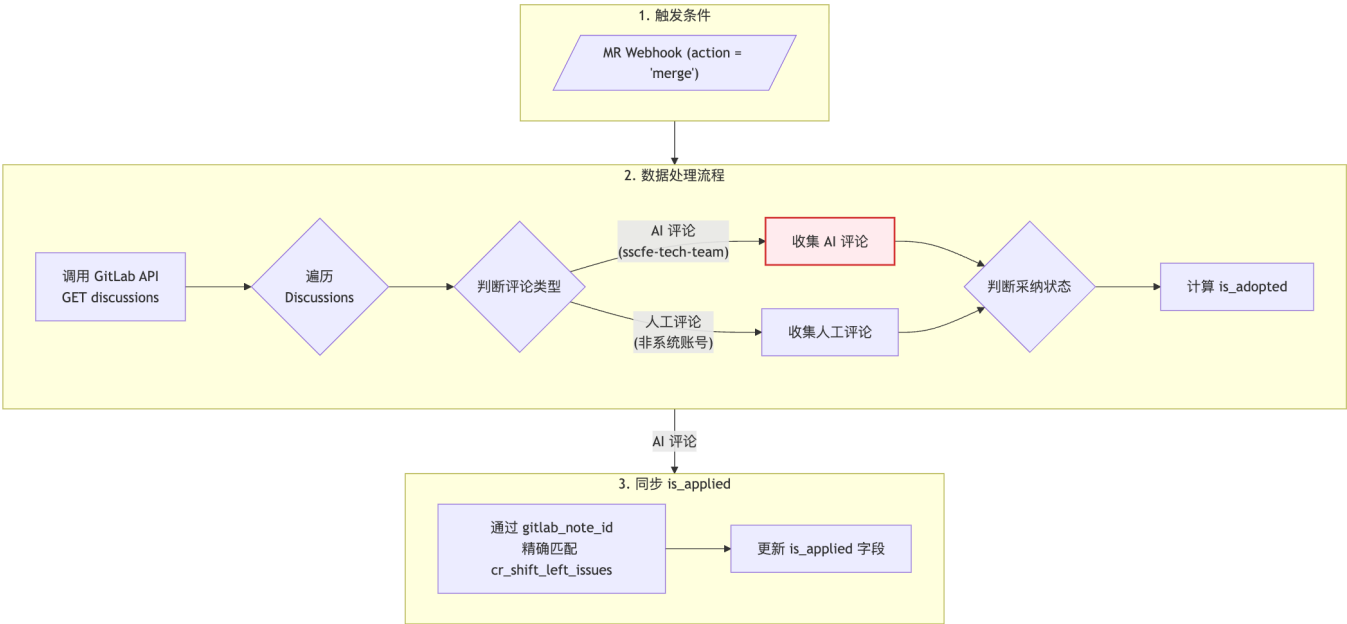
|                     |          |
|---------------------|----------|
| adoption_checked_at | 采纳状态检测时间 |
|---------------------|----------|

## MR 模式

AI CR MR 模式的自动采纳判断逻辑与人工评论采纳识别原理相同，参考人工 CR 内容。

## 数据流程图

与人工 CR 采纳判断流程差异点为红色模块，增加了 AI 评论的判断入口。



## 数据模型

在 cr\_shift\_left\_issues 表新增 gitlab\_note\_id 字段：

```

1 ALTER TABLE cr_shift_left_issues
2 ADD COLUMN gitlab_note_id BIGINT DEFAULT NULL COMMENT 'GitLab 评论 note_id,
  用于精确匹配';
3
4 -- 添加索引
5 CREATE INDEX idx_gitlab_note_id ON cr_shift_left_issues(gitlab_note_id);

```

cr\_shift\_left\_mrs.mr\_comments: 存储所有评论（人工 + AI），通过 comment\_type 区分，仅用于留档

```

1 interface MrComment {
2   // GitLab 标识
3   note_id: number;
4   // 采纳状态
5   is_adopted: number;           // 1-采纳, 0-未采纳
6
7   // 评论类型
8   comment_type: 'manual' | 'ai'; // 新增字段
9
10  ...
11 }

```

## 人工 CR 采纳数据收集

### 采纳判断方案

由于项目配置了 "所有 Discussion 必须 Resolve 后才能 Merge" 的规则，导致：

- 所有评论在 Merge 前都会被 Resolve
- 但 Resolve 不代表真的修改了代码（可能只是点了 Resolve 按钮）
- 因此不能单独依赖 Resolve 状态来判断采纳

当评论所在的代码行被修改后，GitLab 会自动在该 Discussion 的 notes 中添加一条系统消息：

```

1 {
2   "id": 26844171,
3   "type": "DiffNote",
4   "body": "changed this line in [version 3 of the diff](/xxx)",
5   "system": true,
6   "resolvable": false
7 }

```

采纳判断方案：基于 GitLab 系统消息

采纳判断逻辑：

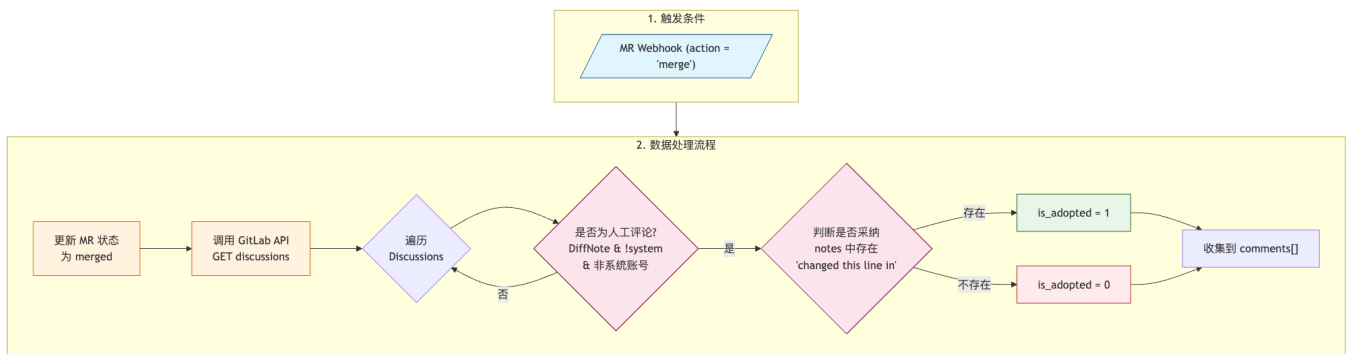
1 人工 CR 采纳 = Discussion 中存在 "changed this line in" 的系统消息（且 MR 被 merged）

|     |                                                                                                                   |
|-----|-------------------------------------------------------------------------------------------------------------------|
| 采纳  | Discussion 的 notes 中存在 <code>system: true</code><br>且 <code>body</code><br>包含 <code>"changed this line in"</code> |
| 未采纳 | MR 已 merged, 但 Discussion 中没有上述系统消息                                                                               |

AI 评论特征：

- `author.username === 'sscfe-tech-team'` （AI CR 发起账号）
- `type === 'DiffNote'` （行间评论）
- `system === false` （非系统消息）

## 数据流程图



# GitLab Discussions API 返回数据分析

API 响应结构:

▼ Plain Text |

```
1 GET /projects/:id/merge_requests/:iid/discussions
```

关键字段说明:

|                 |                                               |
|-----------------|-----------------------------------------------|
| id              | Discussion 唯一标识                               |
| individual_note | true 表示独立评论（非 DiffNote），false 表示 DiffNote 讨论串 |
| notes           | 评论数组，包含原始评论和后续回复/系统消息                         |
| notes[].type    | "DiffNote" 表示行内评论                             |
| notes[].system  | true 表示系统自动生成的消息                              |
| notes[].body    | 评论内容，系统消息会包含特定格式                              |

[接口返回示例](#)

## 数据模型设计

1. 扩展 cr\_shift\_left\_mrs 表

▼ JavaScript |

```
1  -- 新增字段
2  ALTER TABLE cr_shift_left_mrs
3  ADD COLUMN mr_comments JSON DEFAULT NULL COMMENT '人工 CR 评论列表';
4
5  ALTER TABLE cr_shift_left_mrs
6  ADD COLUMN manual_cr_total INT DEFAULT NULL COMMENT '人工 CR 评论总数';
7
8  ALTER TABLE cr_shift_left_mrs
9  ADD COLUMN manual_cr_adopted INT DEFAULT NULL COMMENT '人工 CR 采纳数';
```

2. mr\_comments 字段结构

```

1  ▾ interfaceMrComment {
2      // GitLab 标识
3      note_id: number;           // GitLab note ID (唯一标识)
4      discussion_id: string;     // Discussion ID (用于关联系统消息)
5
6      // 评论人信息
7      author_username: string;   // 评论人用户名
8      author_email: string;     // 评论人邮箱
9
10     // 评论位置
11     file_path: string;         // 文件路径
12     line_number: number;      // 行号
13
14     // 评论内容
15     content: string;          // 评论内容 (截断存储, 最多 500 字符)
16
17     // 采纳状态
18     is_adopted: number;        // 1-采纳 (有代码修改), 0-未采纳
19
20     // 时间
21     commented_at: string;      // 评论时间 (ISO 8601)
22 }

```

[mr\\_comments](#) 内容示例

## Bug 数据收集

### 数据收集

- 数据来源: DMS Bug API – `https://jira.shopee.io/rest/api/2/issue/:jira_id`
- 数据落库: `cr_shift_left_bugs` 表
- 触发时机: Feature 分支合入 UAT 时

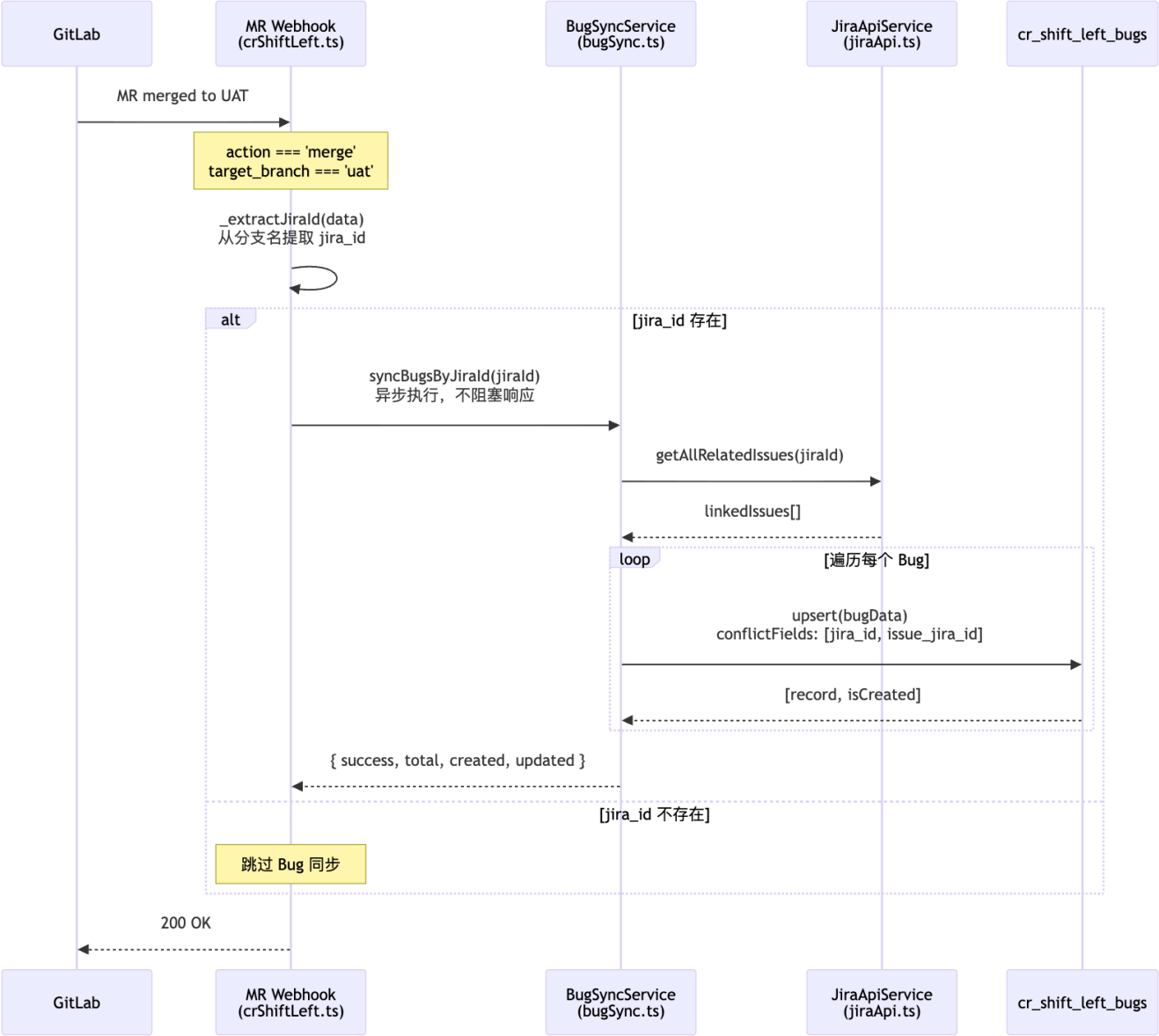
### Bug 数据结构分析

[参考 QA 提供的 Bug 规范](#)

### 存储表结构设计

[根据 Jira 字段, 映射后](#)

## Bug 同步流程



## 数据清理

从 Bug 规范中，我们可以获取到的字段仅有：

1. `status`：只有 Done 状态的 Bug 才纳入统计
2. `components`：归属 FE/BE 并不影响是否纳入统计，只影响团队管理的数据统计
3. `bug_reason`：问题类型，关键的字段
4. `bug_type`：仅用于区分是否为 QA 提的问题，而且填写不规范会导致统计出错，不考虑使用

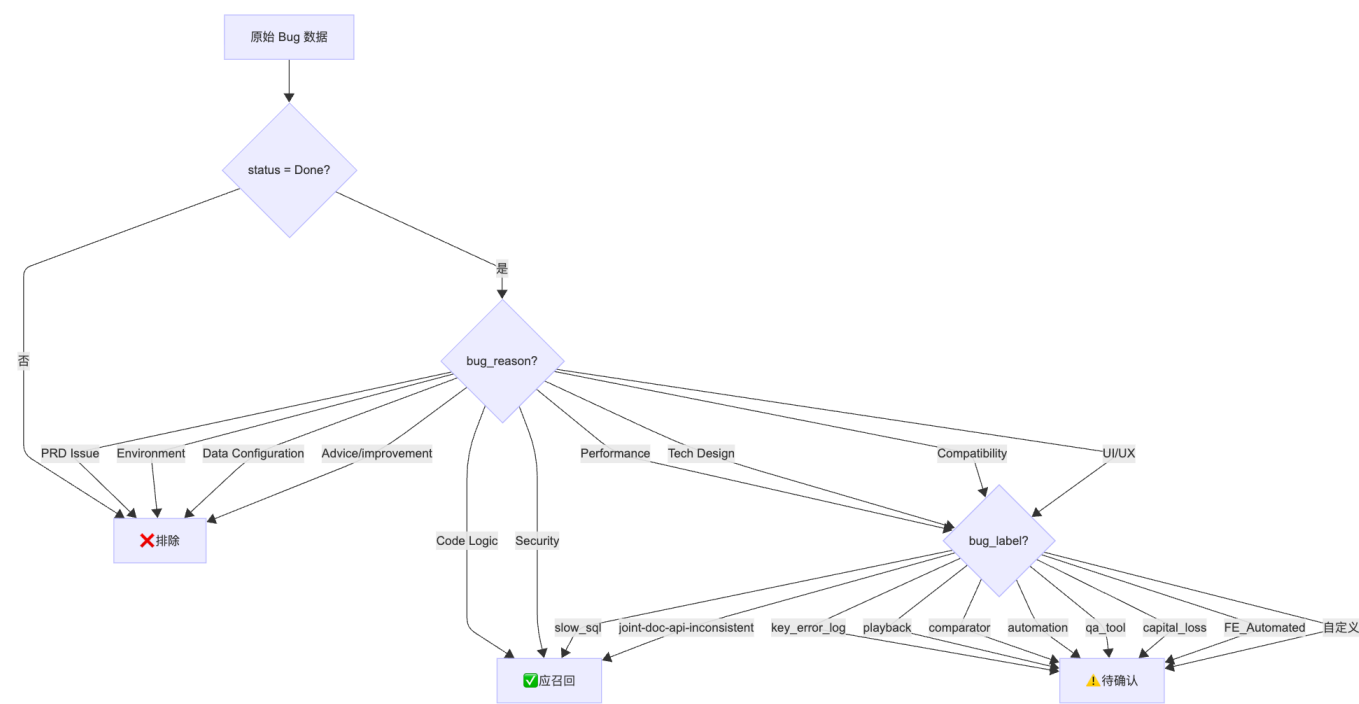


## 5. bug\_label: 细分类型, 关键的字段

|                            |                 |                    |
|----------------------------|-----------------|--------------------|
| status                     |                 |                    |
| To Do                      | ❌ 排除            | 未开始处理              |
| Doing                      | ❌ 排除            | 处理中                |
| Closed                     | ❌ 排除            | 已关闭 (可能是无效 Bug)    |
| Icebox                     | ❌ 排除            | 冻结状态               |
| bug_reason                 |                 |                    |
| Security                   | ✅ 应召回           | 安全问题, CR 必须关注      |
| PRD Issue                  | ❌ 不召回           | 需求问题, 非 CR 范畴      |
| Environment                | ❌ 不召回           | 环境问题 (DB 连接、网络等)   |
| Data Configuration         | ❌ 不召回           | 数据配置问题             |
| Advice/improvement         | ❌ 不召回           | 需求优化建议, 非 Bug      |
| Performance                | ⚠️ 需看 bug_label | 部分性能问题可在 CR 发现     |
| Tech Design                | ⚠️ 需看 bug_label | 设计问题, 需看具体场景       |
| Compatibility              | ⚠️ 需看 bug_label | 兼容性问题, 部分可在 CR 发现  |
| UI/UX                      | ⚠️ 需看 bug_label | 界面/体验问题, 仅部分可召回    |
| bug_label                  |                 |                    |
| joint-doc-api-inconsistent | ✅ 应召回           | 接口联调/文档不一致, CR 应检查 |

|               |        |              |
|---------------|--------|--------------|
| key_error_log | ⚠️ 待确认 | panic, 运行时问题 |
| playback      | ⚠️ 待确认 | 流量回放问题       |
| comparator    | ⚠️ 待确认 | 流量比对         |
| automation    | ⚠️ 待确认 | 自动化覆盖问题      |
| qa_tool       | ⚠️ 待确认 | 测试工具问题       |
| capital_loss  | ⚠️ 待确认 | 资损类          |
| FE_Automated  | ⚠️ 待确认 | 前端 UI 工具类    |
| 自定义           | ⚠️ 待确认 | 用户自定义 label  |

### 清理流程



### 逻辑决策树

```

1 Bug 数据
2 |
3 |— status ≠ "Done"→ ❌ 排除 (未修复的 Bug 不统计)
4 |
5 |— status = "Done"
6 |
7 |   |— bug_reason = "Code Logic"→ ✅ 应召回
8 |   |— bug_reason = "Security"→ ✅ 应召回
9 |
10 |   |— bug_reason = "PRD Issue"→ ❌ 不召回
11 |   |— bug_reason = "Environment"→ ❌ 不召回
12 |   |— bug_reason = "Data Configuration"→ ❌ 不召回
13 |   |— bug_reason = "Advice/improvement"→ ❌ 不召回
14 |
15 |   |— bug_reason ∈ {Performance, Tech Design, Compatibility, UI/UX}
16 |   |
17 |   |— 检查 bug_label
18 |   |
19 |   |   |— bug_label = "slow_sql"→ ✅ 应召回
20 |   |   |— bug_label = "joint-doc-api-inconsistent"→ ✅ 应召回
21 |   |
22 |   |   |— bug_label = "key_error_log"→ ⚠ 待确认
23 |   |   |— bug_label = "playback"→ ⚠ 待确认
24 |   |   |— bug_label = "comparator"→ ⚠ 待确认
25 |   |   |— bug_label = "automation"→ ⚠ 待确认
26 |   |   |— bug_label = "qa_tool"→ ⚠ 待确认
27 |   |   |— bug_label = "capital_loss"→ ⚠ 待确认
28 |   |   |— bug_label = "FE_Automated"→ ⚠ 待确认
29 |   |   |— bug_label = "自定义"→ ⚠ 待确认

```

## 清理结果分类

|     |                     |          |
|-----|---------------------|----------|
| 应召回 | 明确是代码问题, CR<br>应该发现 | 计入 FN    |
| 不召回 | 明确非代码问题             | 不计入 FN   |
| 待确认 | 无法自动判断              | 人工复核后再判断 |

## 判定规则配置

为了兼容外部字段的扩展以及规范变更等需求，我们将采用配置化方式管理判定规则：

```
1  # 召回判定规则配置
2  recall_rules:
3    # 第一层: status 过滤
4    status:
5      include:
6        - "Done"          # 只有 Done 状态才纳入统计
7      exclude:
8        - "To Do"
9        - "Doing"
10       - "Closed"
11       - "Icebox"
12
13   # 第二层: bug_reason 判定
14   bug_reason:
15     # 直接召回
16     recall:
17       - "Code Logic"      # 代码逻辑问题
18       - "Security"        # 安全问题
19
20     # 直接排除
21     exclude:
22       - "PRD Issue"        # 需求问题
23       - "Environment"     # 环境问题
24       - "Data Configuration" # 数据配置问题
25       - "Advice/improvement" # 建议/改进
26
27     # 需要进一步看 bug_label
28     conditional:
29       - "Performance"
30       - "Tech Design"
31       - "Compatibility"
32       - "UI/UX"
33
34   # 第三层: bug_label 白名单 (针对 conditional 的 bug_reason)
35   bug_label:
36     # 明确应召回的 label
37     recall:
38       - "slow_sql"          # 慢 SQL
39       - "joint-doc-api-inconsistent" # 接口联调/文档不一致
40
41     # 待确认的 label (人工复核)
42     pending:
43       - "key_error_log"
44       - "playback"
45       - "comparator"
```

```

46         - "automation"
47         - "qa_tool"
48         - "capital_loss"
49         - "FE_Automated"
50         - "自定义"

```

## 实际 Case 说明

|                              |                            |
|------------------------------|----------------------------|
| <b>Case 1: 代码逻辑问题（应召回）</b>   |                            |
| bug_reason                   | Code Logic                 |
| bug_label                    | -                          |
| summary                      | 列表翻页后数据重复                  |
| <b>Case 2: 慢 SQL 问题（应召回）</b> |                            |
| bug_reason                   | Performance                |
| bug_label                    | slow_sql                   |
| summary                      | 列表查询超时，SQL 未加索引            |
| <b>Case 3: 接口文档不一致（应召回）</b>  |                            |
| bug_reason                   | Tech Design                |
| bug_label                    | joint-doc-api-inconsistent |
| summary                      | 接口返回字段与文档定义不一致             |
| <b>Case 4: PRD 问题（不召回）</b>   |                            |
| bug_reason                   | PRD Issue                  |
| bug_label                    | -                          |
| summary                      | 需求理解偏差导致功能实现错误             |
| <b>Case 5: 环境问题（不召回）</b>     |                            |
| bug_reason                   | Environment                |
| bug_label                    | -                          |
| summary                      | 测试环境数据库连接超时                |

## 人工复核机制

核心问题：谁来做人工复核？

|              |          |                 |                   |     |
|--------------|----------|-----------------|-------------------|-----|
| 方案 A：Bug 经办人 | assignee | 最了解 Bug 上下文     | 增加开发负担，可能不配合      | 低   |
| 方案 B：QA      | reporter | 提 Bug 的人最清楚问题性质 | QA 不一定了解 CR 能发现什么 | 中   |
| 方案 C：CR 工具运营 | 工具团队     | 专业、一致性高         | 人力有限，不了解业务细节      | 中   |
| 方案 D：AI 辅助判断 | LLM      | 无人力成本           | 准确性待验证            | 可探索 |
| 方案 E：不复核     | —        | 无成本             | 待确认的 Bug 不计入召回率   | 高   |

轻量级复核方案

|                  |                                                                                                                                                                            |
|------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 方案 A：嵌入 Bug 关闭流程 | <p>在 Bug 关闭时，增加一个字段让 assignee 填写：</p> <p>这个 Bug 是否可以在 Code Review 阶段发现？</p> <ul style="list-style-type: none"><li>• 是，CR 应该能发现</li><li>• 否，CR 无法发现</li><li>• 不确定</li></ul> |
| 方案 B：抽样复核        | <ul style="list-style-type: none"><li>• 每周抽取 10–20 个待确认 Bug</li><li>• 由工具运营团队复核</li><li>• 用于验证规则准确性，而非全量复核</li></ul>                                                       |

## 数据归属

由于在上面实现

在做真实问题的召回时，我们只能关注前后端的代码相关的问题，也就是说，非代码相关的问题（如责任方在 PM、或外部环境），在清理数据时，是需要被过滤掉的。

判定规则

纯文本匹配

|   |                                                        |         |
|---|--------------------------------------------------------|---------|
| 1 | 包含完整单词 PM<br>/ 产品<br>/ FPM<br>/ BPM<br>/ 需求变更<br>/ PRD | PM      |
| 2 | 包含完整单词 FE<br>/ 前端                                      | FE      |
| 3 | 包含完整单词 BE<br>/ 后端<br>/ 流量回放                            | BE      |
| 4 | bug_reason = UI/UX                                     | FE      |
| 5 | labels 包含完整单词 FE                                       | FE      |
| 6 | labels 包含完整单词 slow_sql<br>/ joint-doc-api-inconsistent | BE      |
| 7 | 未命中以上任何条件的记录                                           | unknown |

通过 assignee 兜底

对于仍为 unknown 或 PM 且 recall\_status = should\_recall 的 Bug:

- 通过 assignee 邮箱查询 cr\_mcp\_user\_tab 表
- 如果用户角色是 FE 或 BE，则使用该角色

Case 说明

|                                 |  |
|---------------------------------|--|
| Case 1: FE 归属 (summary 包含 [FE]) |  |
|---------------------------------|--|



|                                         |                                                                                      |
|-----------------------------------------|--------------------------------------------------------------------------------------|
| summary                                 | [FE] detail详情页 MECE 字段值未展示 以及存在上传的文件有异常时直接报错                                         |
| components                              | ["sbs-charge"]                                                                       |
| bug_reason                              | Code Logic                                                                           |
| labels                                  | []                                                                                   |
| assignee                                | ran.wang@shopee.com                                                                  |
| <b>Case 2: BE 归属 (summary 包含 【BE】 )</b> |                                                                                      |
| summary                                 | 【BE】 cage 推荐逻辑有误                                                                     |
| components                              | ["SPX-Inner&Outbound"]                                                               |
| bug_reason                              | Code Logic                                                                           |
| labels                                  | []                                                                                   |
| assignee                                | xuan.qiao@shopee.com                                                                 |
| <b>Case 3: PM 归属 (summary 包含 FPM)</b>   |                                                                                      |
| summary                                 | [FPM->PDA] 当订单一个都没分，没有绑定格口号时，incomplete，需要绑定异常basket，提示不友好                           |
| components                              | []                                                                                   |
| bug_reason                              | Code Logic                                                                           |
| labels                                  | []                                                                                   |
| assignee                                | yiting.shi@shopee.com                                                                |
| <b>Case 4: unknown 归属 (无明确标识)</b>       |                                                                                      |
| summary                                 | 【改为从third_party_status_mtime字段获取】 对于延迟添加的场景，Item Order Pool页面的Status Update Time时间不对 |
| components                              | ["ASM"]                                                                              |
| bug_reason                              | Code Logic                                                                           |

|                                   |                                                                  |
|-----------------------------------|------------------------------------------------------------------|
| labels                            | [[]]                                                             |
| assignee                          | <a href="mailto:ally.zhang@shopee.com">ally.zhang@shopee.com</a> |
| <b>Case 5: BE 归属 (labels 不为空)</b> |                                                                  |
| summary                           | 接口 Panic - nil 类型断言                                              |
| components                        | ["Replenishment"]                                                |
| bug_reason                        | Code Logic                                                       |
| labels                            | ["slow_sql"]                                                     |
| assignee                          | <a href="mailto:dev@shopee.com">dev@shopee.com</a>               |

## 边界场景处理

当前数据分布 (应被召回的 Bug) :

|           |             |             |                |
|-----------|-------------|-------------|----------------|
| BE        | 606         | 47.1%       | ✅ 计入召回率        |
| FE        | 427         | 33.2%       | ✅ 计入召回率        |
| unknown   | 190         | 14.8%       | ⚠️ 不计入召回率      |
| PM        | 64          | 5.0%        | ⚠️ 不计入召回率      |
| <b>总计</b> | <b>1287</b> | <b>100%</b> | FE+BE 覆盖 80.3% |

边界场景说明:

|                      |          |            |
|----------------------|----------|------------|
| assignee 不在用户表       | 外部人员或未注册 | 保持 unknown |
| assignee 的 role 是 PM | 用户角色非研发  | 保持 PM      |

|             |       |               |
|-------------|-------|---------------|
| Bug 无任何角色标识 | 填写不规范 | 保持<br>unknown |
|-------------|-------|---------------|

统计时的处理：

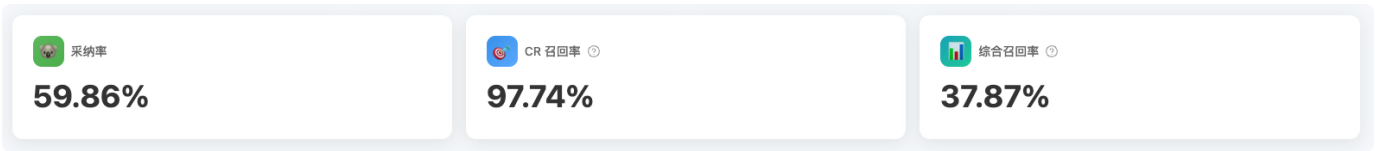
- 召回率统计只关注 FE/BE 角色的 Bug
- PM 和 unknown 的 Bug 不计入召回率分母
- 这样可以更准确地反映 CR 对研发代码问题的召回能力

## 遗留问题

- BE 团队下的人工 Comment 数据未统计上来
  - 原因：由于统计数据依赖 MR 访问权限，而大多 BE 团队接入未配置
  - 解决思路：通过后续代码分析项目，推动 SSC BE 团队的全部仓库接入 IBS AI CR 配置
- unknown/PM 归属的 Bug 未纳入召回率统计
  - 原因：Bug 填写不规范，无法准确判断归属角色（FE/BE）
  - 解决思路：推动 QA 在填写 Bug 时按照规范添加角色标识，例如 components 字段必须填写对应的团队、summary 必须使用【FE】、【BE】、【PM】等关键字

## 展示页面

所有需求：



区分团队：

| 维度     | 名称 | 用户邮箱 | 活跃用户邮箱 | 对话总数  | 发现问题数 | 高危问题数 | 代码总行数     | 采纳率 | CR 召回率 | 综合召回率 | 覆盖需求情况  |
|--------|----|------|--------|-------|-------|-------|-----------|-----|--------|-------|---------|
| + role | FE | 95 人 | 89 人   | 1,574 | 7,799 | 5,001 | 2,947,401 | 56% | 96%    | 25%   | 252 个需求 |
| + role | BE | 72 人 | 60 人   | 435   | 3,997 | 2,437 | 1,201,044 | 67% | 100%   | 47%   | 134 个需求 |
| + role | QA | 2 人  | -      | 0     | 0     | 0     | 0         | -   | -      | -     | -       |

## 参考

Test Bug 规范

<https://zh.wikipedia.org/wiki/F-score>

